Jana Architecture

Jana is a personal tutor and study companion for adults, built on Meta's models. A learner brings their own material or follows a curriculum. Jana teaches from it, checks the work, remembers the learner and keeps saved study working when the connection drops.

This document starts with what Jana can do and then opens each part: what the learner gets, which components do the work, how they connect, and the code that implements it.

CONTENTS

1 What Jana can do
2 System overview
3 An illustrative workflow
4 Agent orchestration
5 Checking, the board and photos
6 Study: from source to checked quiz
7 Your tutor and learner memory
8 Curriculum
9 How Meta's models are integrated
10 Spending controls
11 Privacy and storage boundaries
12 Where it runs
13 Security and quality
14 Status and limits
A Code map
B Glossary

Two minutes

Read the four rules below and the capability map in Part 1.

Fifteen minutes

Add Part 2 for the system, Part 4 for how the agents coordinate and Part 9 for how Meta's models are integrated.

Building or reviewing it

Read everything. Parts cite the files, excerpts and tests behind them, and Part 14 lists the limits.

RULE

Code decides, the model phrases

The engine sets the step, holds the hint level and renders the facts. A model drafts words and chooses tools inside that step.

RULE

Nothing is said until it passes

A reply stays private until its gates pass. A number must trace to a check, the question or the learner.

RULE

A verdict comes from a key or a tool

What could not be checked is labelled "Judged, not checked" and is never counted as mastery.

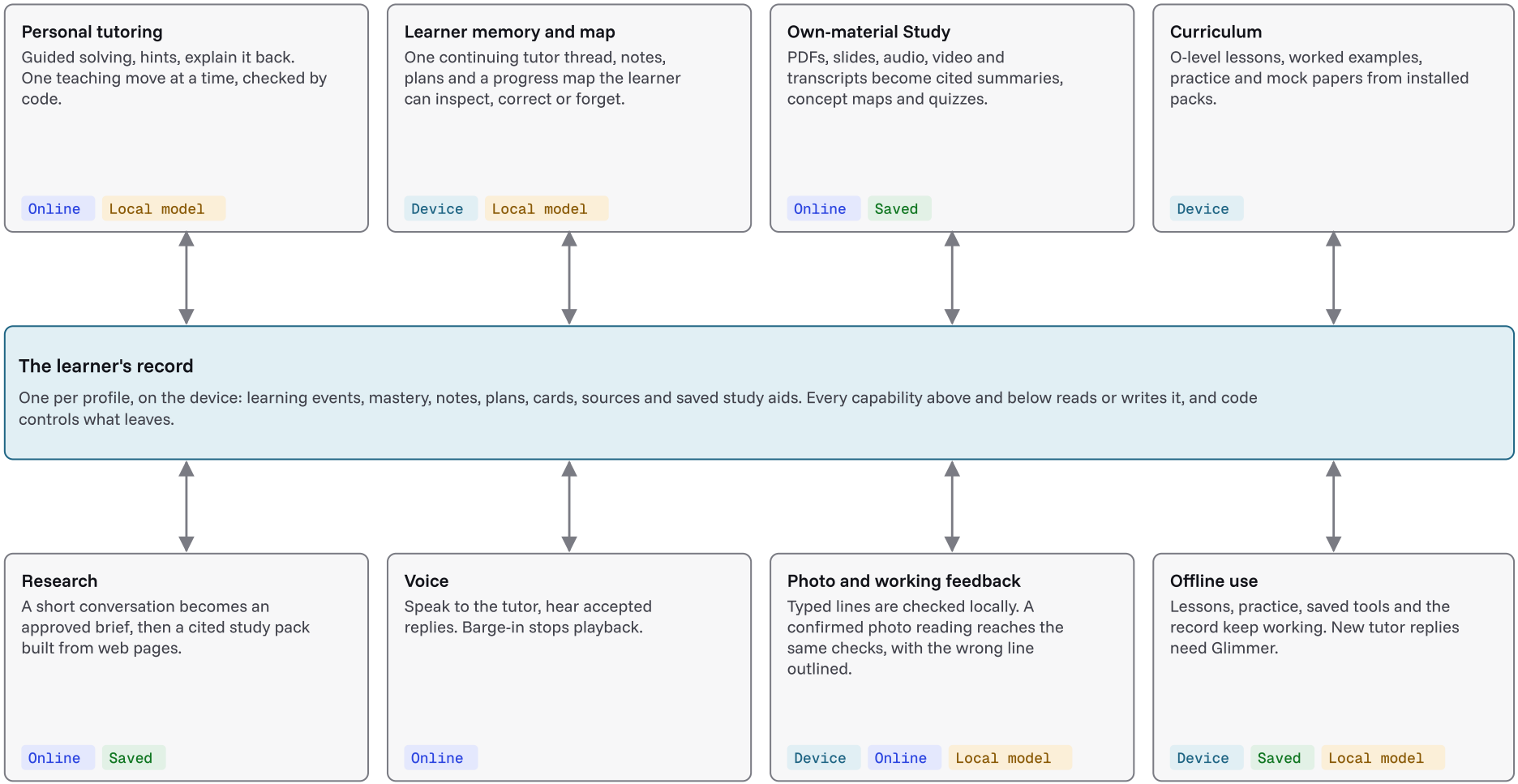
RULE

The record stays with the learner

Learner data is stored on the device. Each kind of data that goes online has its own consent, and the record of what left is kept.

1 What Jana can do

Eight capabilities share one learner's record. The labels show where each one runs: Device means it runs locally, Online needs a connection, Saved stays usable offline once made, and Local model needs Muse Glimmer.



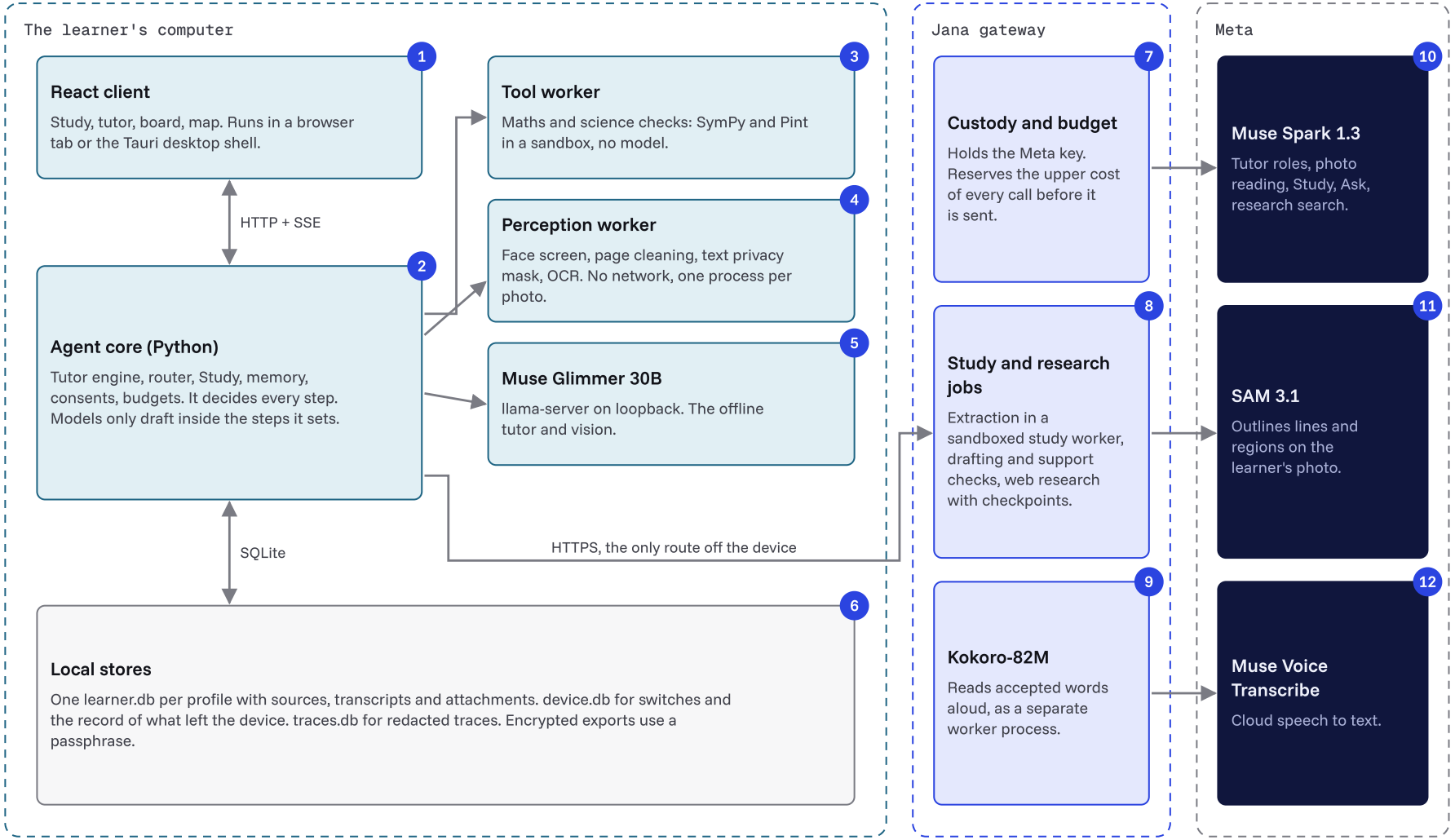
Muse Spark provides the online route and Muse Glimmer the local one. Where both labels appear, either route can serve it. The table below is covered by `test_m7b_offline.py` and `test_m25b_offline_and_privacy.py`.

What works where

WHAT THE LEARNER DOES	ONLINE	CONNECTION LOST, NO LOCAL MODEL	CONNECTION LOST, MUSE GLIMMER INSTALLED
Installed lessons, practice and mock papers	Yes	Yes, marked on the device	Yes, marked on the device
Saved quizzes, flashcards, concept maps and source pages	Yes	Yes	Yes
Checking typed working on the board	Yes	Yes, the tool worker is local	Yes
A new tutor reply	Muse Spark	Authored hints only	Muse Glimmer: 22 to 122 s a reply on an Apple M4 Max, 23 to 49 s on a 24 GB NVIDIA L4. Too slow to use on CPU alone
Reading a photo of working	Muse Spark, with SAM 3.1 outlines	No	Muse Glimmer vision, 37 to 85 s on an Apple M4 Max and 43 s on an NVIDIA L4, with no outlines
New study tools, uploads, research	Yes	Waits for the internet	Waits for the internet
Speaking to the tutor	Voice Transcribe and Kokoro-82M	No	Only with a separate local speech pack, which is partial
Starting the app from cold	Yes	Yes, with the local gateway process running	Yes, tested with the network cut

2 System overview

Most of Jana runs on the learner's computer. One gateway holds the Meta key and the spending limits, and it is the only route to Meta's hosted models.



The client reaches the core over loopback with a launch token. Workers take JSON lines over stdio, and Glimmer is local HTTP. Teal is the device, blue is Jana's gateway, dark is Meta. In the hosted deployment the same core runs on a server (Part 12).

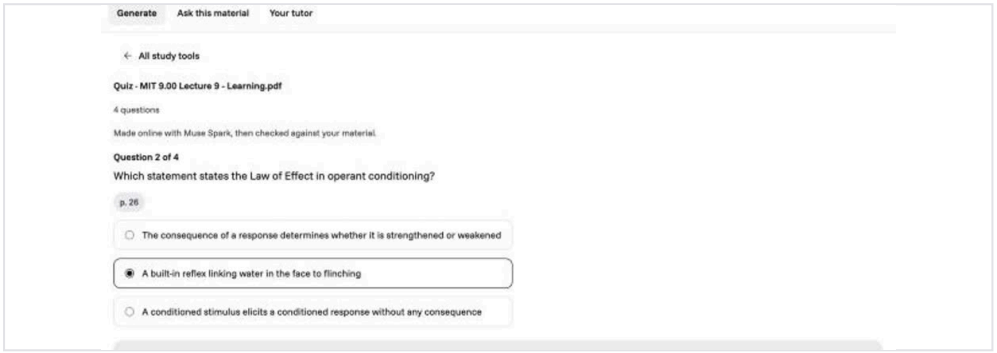
#	PART	WHAT IT DOES	STATUS
1	React client	Everything the learner sees, shared by the browser host and the desktop shell.	Built, tested
2	Agent core	The tutor engine, Study, memory, router and stores. Holds the rules in this document.	Built, tested
3	Tool worker	36 checking functions. Own parser, no `eval`, no network, no model. A timeout answers "unknown".	Built, tested
4	Perception worker	Prepares photos in a disposable sandboxed process. Fails closed without a sandbox.	Built, tested
5	Muse Glimmer 30B	The local model, started on loopback with its vision projector and draft model when present. Text tutoring, guided tutoring and photo reading are verified on an Apple M4 Max and a 24 GB NVIDIA L4 (Part 3). It needs about 21 GB of GPU memory, and a packaged release does not exist yet.	Verified on M4 Max and NVIDIA L4
6	Local stores	Per-learner SQLite and files. Not encrypted at rest: see Part 11.	Built, tested
7	Custody and budget	The only holder of the Meta key. Every call is priced and held before it is sent.	Built, tested
8	Study and research jobs	Long jobs, resumable. Extraction runs network-denied.	Built, tested
9	Kokoro-82M	Speech synthesis, from outside Meta. Reads accepted words, so speech cannot say what the gates refused.	Built, tested
10 to 12	Meta's hosted models	Muse Spark, SAM 3.1 and Voice Transcribe, reached only through the gateway.	Built, tested

WHERE IT LIVES `apps/desktop` `packages/ui/src` `services/agent-core/agent_core` `services/cloud-gateway` `services/tool-worker` `services/perception-worker` `services/study-worker`

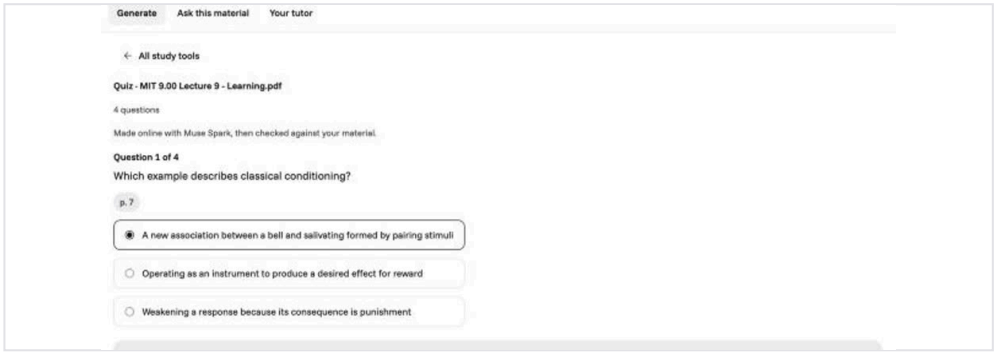
3 An illustrative workflow

This is one representative path through the application: a learner takes a lecture from upload to saved study, then works a problem on paper. Each row shows what the interface, core, workers and models do.

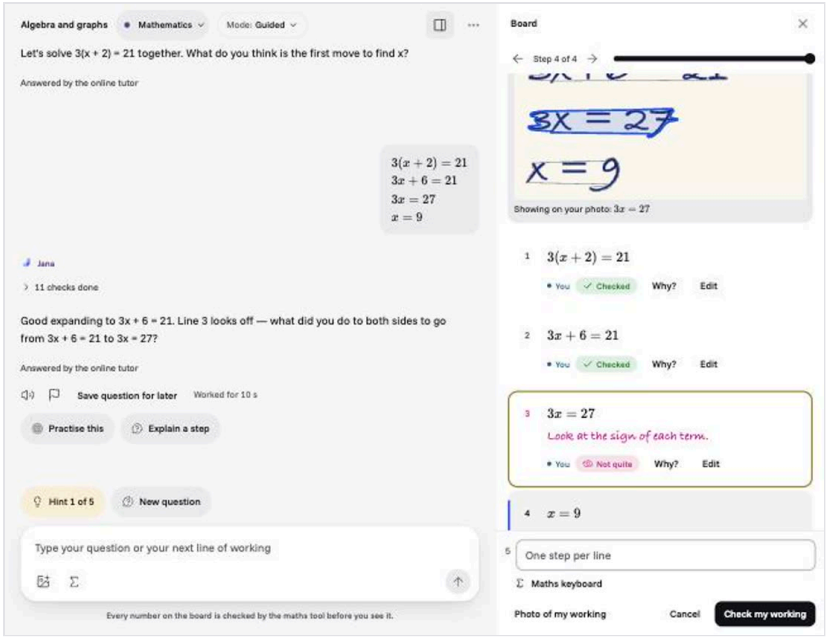
	Add a lecture `needs connection`	**Map and explanation** `generated online, then saved`	**Check the source** `saved`	**Quiz: mistake, then correction** `generated online, marked locally`	**Connection drops** `saved`
`LEARNER`	Uploads a 54-page lecture PDF.	Opens a concept on the map and reads the explanation.	One click opens the original slide beside the cited passage.	Picks a wrong answer, is told it is incorrect with the answer held back, then tries again.	Answers another saved question, then reopens a slide and the map.
`INTERFACE AND CORE`	Type and size checks, chunks with page locators, consent check before anything leaves.	Validates the map's structure. Every quote must equal a span in the source.	Opens the stored page and span.	Compares the choice to the key.	Marks the saved quiz on this computer, with no new cloud call.
`WORKERS AND MODELS`	The gateway's study worker decodes the file, with no network. No model.	Muse Spark drafts. A separate call checks support.	No model.	Muse Spark generated the questions earlier. Marking uses no model.	No model.



The mistake. Marked incorrect, answer held back, hint offered.



The correction. An explanation cited to page 7, labelled "Judged, not checked" because no tool can prove it.



Working on paper. On a curriculum subject, lines 1 and 2 are checked, line 3 is flagged and outlined on the learner's photo, and the tutor asks a question instead of giving the answer.

Captures from the running application on 4 November, with invented adult learners and a prepared photo. Lecture: John Gabrieli, MIT 9.00SC Introduction to Psychology, Fall 2011, Lecture 9, CC BY-NC-SA. No MIT affiliation or endorsement.

What each observation is

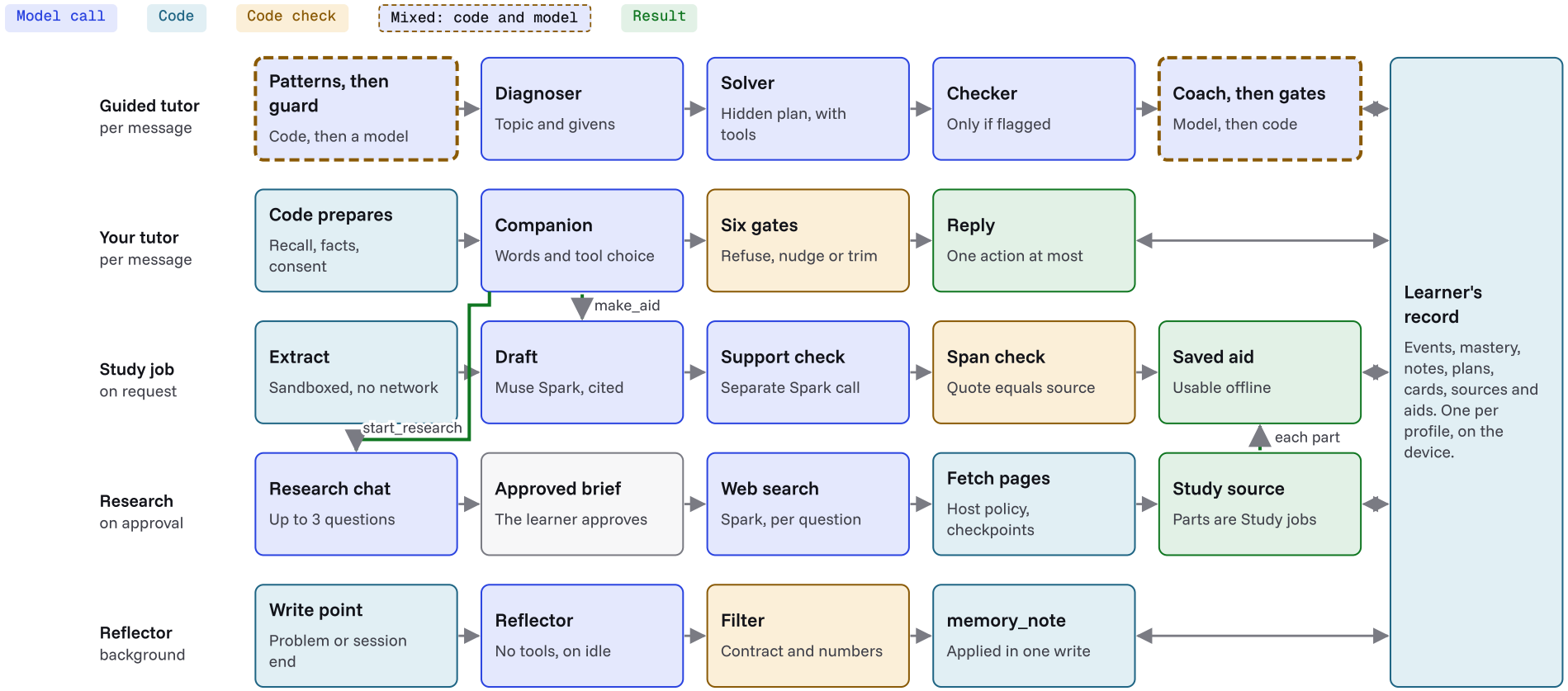
PATH	KIND	OBSERVED
Saved quiz, sources and map, gateway paused and Wi-Fi on	`Saved data`	Worked, with local quiz marking and no new cloud call.
Your tutor on Muse Glimmer, Apple M4 Max with 128 GB, no cloud provider configured	`Real model`	First run: 12 of 12 complete answers, 24.52 s to 122.53 s, median 84.3 s. It showed two weaknesses, both since fixed: after a restart the tutor said it could not see the first message, and one reply added unrelated due-card counts. Retest with the fixes in this revision: 9 of 9 complete, 21.9 s to 122 s, median 46.6 s, across a real restart. The first question was named correctly and due counts appeared only when asked for.
Your tutor on Muse Glimmer, Linux VM with a 24 GB NVIDIA L4	`Real model`	With the fixes in this revision: 9 of 9 complete, 22.8 s to 49 s, median 34.9 s, across a real restart. Peak GPU memory was 20.7 GB. One follow-up, "what did I ask just before that?", got a doubtful answer.
Guided tutoring on Muse Glimmer	`Real model`	Physics question, with the fixes in this revision. On the NVIDIA L4: new question 68 s (Diagnoser 11 s, Solver 38 s), wrong line 49 s, correct final line 15 s. On the M4 Max: new question 76 s, wrong line 109 s, correct final line 62 s. A correct chained line, `a = 20 / 4 = 5 m/s^2`, closes the question. A wrong one, `a = 20 / 2 = 10 m/s^2`, is marked wrong by code with a question back.
Photo reading on Muse Glimmer	`Real model`	A prepared solution of 4 lines read correctly and nothing left the device. M4 Max: 84.88 s with line boxes, 37.13 s without. NVIDIA L4: 42.6 s with line boxes. This ran through the engine, not the capture screen, and real handwriting is not tested.
Cold start with no network, Linux VM with 8 vCPUs and 32 GB	`Real network cut`	With a firewall rule cutting all outbound traffic, and Meta, Hugging Face and GitHub confirmed unreachable, the gateway, core, client and Muse Glimmer started in 132 s. A new learner could open practice, and answers were marked. The local gateway process is still required.
Muse Glimmer with no GPU, Linux VMs	`Real model`	CPU alone is not usable, at 2.1 tokens a second on 8 vCPUs. Before the fix a "Your tutor" message had no reply after 25 minutes. With the fixes in this revision, the GPU hidden, the learner got a plain "taking too long" line at 242 s, and the trace records `local_time_limit`.
Earlier Muse Glimmer runs, before these fixes	`Real model`	Mixed: stalled attempts, source-retrieval misses and a reply that offered to pull pages instead of answering. These fixes do not prove every retrieval case is resolved.

PATH	KIND	OBSERVED
Packaged desktop app, macOS	`Real model`	A debug Tauri build reached a local reply on a throwaway profile, in 92 s, using a test-signed model manifest and a manual override. The first question got only a fallback line after 138 s. A release build opens, but its core refuses to start without an agent bundle signed under a real release root.
Photo of working with SAM 3.1, a prepared image	`Real model`	With Muse Spark and the gateway: outlines, wrong line and clue worked. Real handwriting through a real file picker is not tested.
Hosted deployment with Muse Spark	`Real model`	Upload 2 s, summary 14 s, quiz 33 s.
Falling back to local and returning	`Simulated failure`	Passes in tests and by stopping the gateway by hand. Turning Wi-Fi off on a laptop is not tested.
Multi-day learner journey	`Scripted test`	Real API, storage and marker, a scripted language model, and a clock advanced day by day. Passed, 4 Nov.

WHERE IT LIVES `packages/ui/src/learn/StudyWorkspace.tsx` `packages/ui/src/learn/StudyQuiz.tsx` `packages/ui/src/board/PhotoRegions.tsx` `services/agent-core/tests/test_learning_agent_journey.py` `services/agent-core/tests/test_w3_power_cut.py`

4 Agent orchestration

The engine is the orchestrator. It sequences the tutoring roles, owns the learner's state and controls which model output is shown. Each workflow below runs separately and meets the others only in the learner's record or through an explicit tool.



Five workflows, not one chain. A tool worker, gate or consent check is code and not an agent. Green arrows are the only handoffs between workflows: `make_aid` starts a Study job, `start_research` only opens the research conversation, and each part of a research pack is an ordinary Study job.

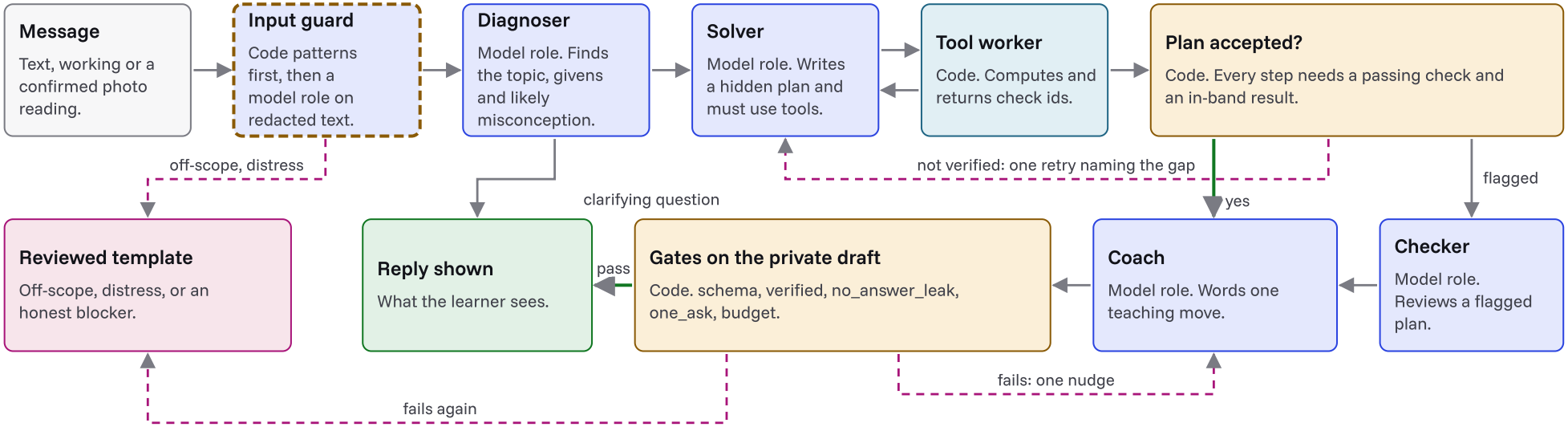
The model roles

ROLE	RESPONSIBILITY	TRIGGER	INPUTS	TOOLS OFFERED	DECISION OR OUTPUT	FILES
Input guard	Screens for off-scope or distress	Each message, after code patterns	Redacted text	None	In scope or not. Out of scope gets a template	`services/agent-core/agent_core/engine/auxiliary.py`

ROLE	RESPONSIBILITY	TRIGGER	INPUTS	TOOLS OFFERED	DECISION OR OUTPUT	FILES
Diagnoser	Finds topic, givens and misconception	New problem, or an unconfirmed answer	Question, candidate topics and skills	`curriculum_lookup`	`diagnosis.schema.json`, or a clarifying question	`services/agent-core/agent_core/engine/team.py`
Solver	Writes the hidden plan	New problem. Starts speculatively beside the Diagnoser	Question and givens	Compute tools, `skill`	`solution-plan.schema.json`. Values must come from tools	`services/agent-core/agent_core/engine/turn.py`
Checker	Reviews what code could not settle	A flagged plan, or an answer code could not confirm	The plan or answer, with check ids	Compute tools, `skill`	`verdict.schema.json`, citing the checks used	`services/agent-core/agent_core/engine/turn.py` `services/agent-core/agent_core/engine/team.py`
Coach	Words one teaching move	Every tutoring reply	Verified plan, step, hint level, memory	`skill`, `content_search`, `memory_recall`, `session_recall`	`tutor-turn.schema.json`, then gates	`services/agent-core/agent_core/engine/turn.py`
Companion	The continuing tutor thread	Each "Your tutor" message	Facts from tools, history, filtered by consent	`memory_recall`, `session_recall`, `study_search`, `progress_read`, `plan`, `make_aid`, `start_research`	`companion-turn.schema.json`, then six gates	`services/agent-core/agent_core/engine/companion.py`
Reflector	Turns activity into notes	End of a problem or session, onboarding, or a request, when idle	Code-built window of events	None. The engine applies the result	`reflection.schema.json`, through `memory_note`	`services/agent-core/agent_core/engine/reflection.py`
Research chat	Agrees what a pack is for	Each research turn, online	Conversation and profile	None	A question or an approved brief. At most 3 questions	`services/agent-core/agent_core/engine/research_chat.py`
Study draft and support	Drafts an aid, then checks support	A Study job	Selected passages	None on the device route	Cited draft, then per-item support	`services/cloud-gateway/cloud_gateway/study.py`
Judge	Marks explanations and written answers	A learner explains or writes	Key points, the answer	None	Per-point verdict, quoting the learner	`services/agent-core/agent_core/engine/assessment/explain.py`
Vision	Reads photos of working	A photo after its notice	The screened image	None	Lines and boxes for the learner to confirm	`services/agent-core/agent_core/engine/vision.py`

One tutoring turn

Model call | Code | Code check | Mixed: code and model | Result



The engine runs the sequence and holds the hint level. The Checker does not run on every turn. It runs when a plan is flagged as implausible, and in parallel with the Diagnoser when code cannot confirm a learner's answer. On the lean local class it is skipped when code has already marked the line wrong. If a plan cannot be verified after one retry, the learner gets an honest blocker in place of an unchecked answer.

```
for attempt in range(2):
    envelope = (await self._speculated(turn) if attempt == 0 else None) or await self.delegate(
        "solver", task, turn
    )
⋮
# Every computed step and the answer carry a passing check from the worker: that is the
# deterministic verification of AGT-10, so the Checker is not delegated to (AGT-13; a model's
# judgment never overrides a worker check).
live_flows.plan_checked(self, True)
return plan
```

`services/agent-core/agent_core/engine/turn.py`, lines 1433–1436, 1450–1454, revision df10b972. A Solver plan with passing checks skips the Checker, because a model never overrides a worker check.

```
value, check_id = await run_tool(engine, turn, "math_check_step",
                                 {"prev": for_tool(before), "next": line, "moves": True})  # fmt: skip
verdict = (value or {}).get("verdict")
```

`services/agent-core/agent_core/engine/slice.py`, lines 240–242, revision df10b972. The engine itself calls the tool on a learner's line. No model is asked.

What a completed task gives the learner

WORKFLOW	ON SUCCESS	ON REFUSAL OR FAILURE
Guided tutor	One checked teaching move, with a board line where relevant	One nudge, then a reviewed template. An unverified plan becomes an honest blocker
Your tutor	A reply with facts from tools and at most one action	One nudge, then "ask me again". A reply that only asks twice keeps its last question
Study job	A saved aid, usable offline, with citations	Unsupported items drop, one bounded redraft. Offline, the job waits and does not bill again
Research	One Study source and its parts as ordinary aids	`no_sources`, `budget`, `timeout`, `upstream`, `interrupted` or `cancelled`
Reflector	Notes and a session summary	A bad output gets one correction, then the job is dropped. A forget cancels and requeues it

A real trace

A Companion turn from a run on 4 November with the cloud connection down, on Muse Glimmer. Traces are redacted and stay on the device.

FIELD	RECORDED
Mode and connectivity	`study_companion`, `offline` at start
Model calls	3 calls, role `companion`, served by `muse-glimmer`
Attempt 1	`one_ask` fired, so the draft got one nudge
Attempt 2	`one_ask` `passed_after_nudge`. `fact_binding`, `search_claim`, `model_claims`, `output_critic` and `fact_refs` passed
Result	`complete`, 53.6 s

Tests

WHAT IS PROVED	TEST	RESULT
Roles and manifests	`tests/test_m14a_roles.py`	Passed, 4 Nov
Gates and nudges	`tests/test_m14b_gates.py`	Passed, 4 Nov
A flagged plan needs the Checker, an in-band plan skips it	`test_ver26_checker_review_clears_a_flagged_plan`, `test_ver26_in_band_plan_skips_the_checker`	Passed, 4 Nov
A chained line is judged by its value and arithmetic, and a lean class skips the Checker on a decided wrong line	`tests/test_chained_line.py`	Passed, 4 Nov
Reflector scheduling and gates	`tests/test_m15e_reflector.py`	Passed, 4 Nov
Research job, checkpoints and resume	`services/cloud-gateway/tests/test_research.py`	Passed, 4 Nov

WHERE IT LIVES `content/agent/roles` `services/agent-core/agent_core/router/router.yaml` `services/agent-core/tests/test_m14a_roles.py` `services/agent-core/tests/test_m14b_gates.py` `services/agent-core/tests/test_m12f_live_plausibility.py` `services/agent-core/tests/test_m15e_reflector.py` `services/cloud-gateway/tests/test_research.py`

5 Checking, the board and photos

A learner's working is checked line by line, by code, against the line before it. Supported steps are checked without requiring a single prescribed solution path, and a step that cannot be checked stays unverified. Photos reach the same checks once the learner has confirmed how Jana read them.

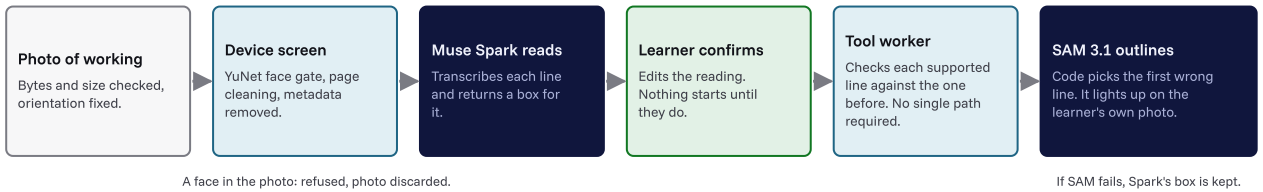


Photo of working. Reading never starts a tutoring turn on its own.

LEARNER BENEFIT

Feedback lands on the exact line, on the learner's own page, without forcing one solution path.

FLOW

Screen the photo, read it, the learner confirms, each line is checked, then the first wrong line is outlined.

COMPONENTS

Client board, perception worker, Muse Spark or Glimmer vision, tool worker, SAM 3.1 through the gateway.

CODE DECIDES, MODEL JUDGES

Code: every verdict, the choice of line, the answer gate. Model: reading the handwriting and proposing boxes, which the learner confirms.

Worked example: real checker output

Output of `math_check_step` on this revision. Two supported methods pass, and wrong lines get a category and a reason. No model is involved.

PREVIOUS LINE	NEXT LINE	VERDICT	REASON GIVEN
`3(x+2) = 21`	`3x+6 = 21`	valid	
`3(x+2) = 21`	`x+2 = 7`	valid	A different method, also accepted
`3(x+2) = 21`	`3x+2 = 21`	invalid	arithmetic: 2 should be 6
`3x+6 = 21`	`3x = 27`	invalid	sign: the sign of 6 is wrong
`3x = 15`	`x = 5`	valid	

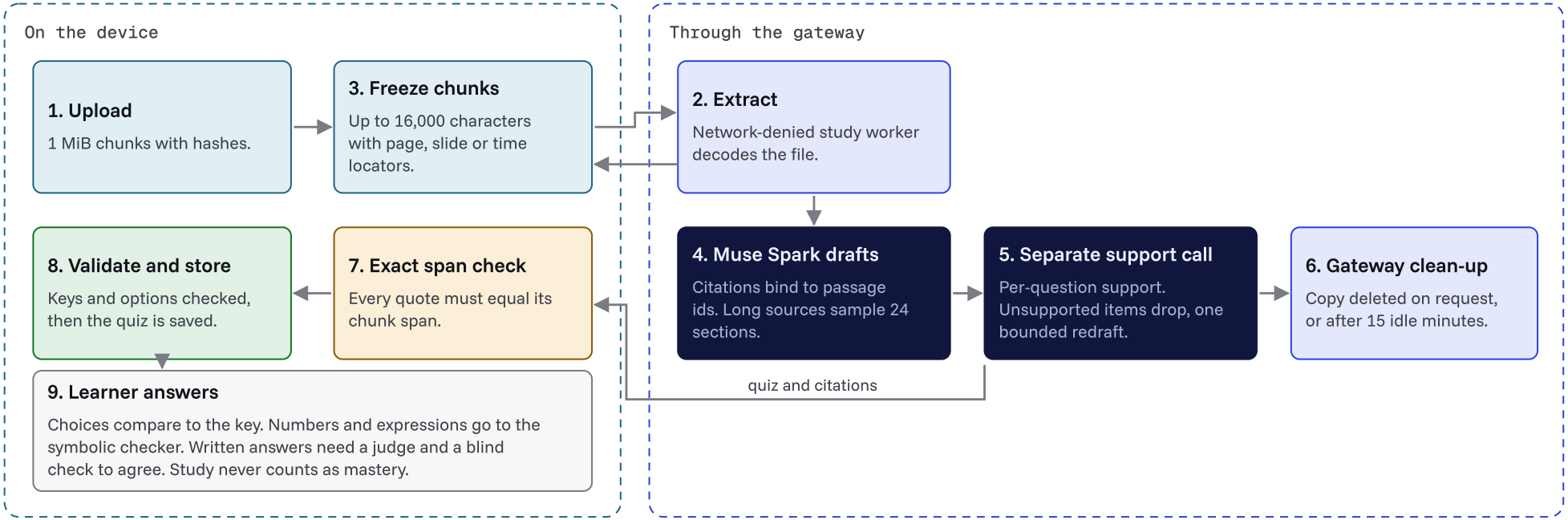
The checker is SymPy and Pint behind its own parser, in a sandboxed process with restricted imports. Each slice has an 800 ms budget and answers "unknown" rather than guessing. An "unverifiable" result creates no mastery evidence. `tests/test_false_accepts.py`, where each case was a reproduced false accept: Passed, 4 Nov.

"Snap your sketch" reads the elements of a drawing and code compares them with the checked figure. It covers motion graphs and free-body diagrams only. The face gate sees faces, not bodies. There is no dedicated local handwriting OCR engine: handwritten working relies on the selected vision model, with the learner confirming the transcription. The local vision path needs Glimmer's projector and a compatible runtime. It read a prepared solution correctly on an Apple M4 Max, and real handwriting is not tested. SAM outlines need connectivity.

WHERE IT LIVES `services/tool-worker/tool_worker/functions/steps.py` `services/agent-core/agent_core/engine/slice.py` `services/agent-core/agent_core/engine/photo_regions.py`

6 Study: from source to checked quiz

A learner adds their own material on any subject and gets study aids cited to it. A draft has to survive a second model check and an exact match against the source before it is saved.



A document becoming a checked quiz. The numbers are the order of events.

LEARNER BENEFIT

Every explanation and question can be traced to the learner's own page, and saved aids stay usable offline.

FLOW

Upload, extract, freeze chunks, draft, support check, span check, save, then mark.

COMPONENTS

Client workspace, core Study desk and queue, gateway Study route, sandboxed study worker, Muse Spark.

CODE DECIDES, MODEL JUDGES

Code: consent, chunking, span match, structure, keys, and what is saved. Model: drafting and judging support, which the same model family does.

Anatomy of a checked quiz question

```
{
  "item_id": "q1",
  "stem": {
    "citations": [{
      "source_id": "src_01M6J4K0W82J...",
      "revision": 1,
      "chunk_id": "sck_01M6J4K1942M...",
      "start": 0, "end": 273,
      "quote": "Conditioned Stimulus (CS) - bell ..."
    }]
  },
  "options": [
    { "option_id": "a", "text": "Learning a new association between a bell and salivating" },
    { "option_id": "b", "text": "Strengthening a response because of its rewarding consequence"
}
  ]
}
// held apart from the view, in "keys"
{ "item_id": "q1", "answer": "a", "marking": "judged" }
// on the aid that holds the quiz
"made_by": { "runtime": "online", "model": "muse-spark-1.3" }
// the learner's attempt
{ "item_id": "q1", "verdict": "correct", "marking": "judged",
  "label": "Judged, not checked" }
```

From a quiz saved during a run on 4 November, with ids and quotes shortened and two options left out.

1. **Traceable to the source.** Each citation names a source revision, a chunk and a character span. This is how "open the original slide" works.
2. **The answer is held apart.** The learner's view has no key, so the interface cannot leak it.
3. **Honest about how it was marked.** `marking` says whether code proved the verdict or it was judged, and the label comes from it. Study never counts as mastery.
4. **Says who made it.** `made_by` records the route and the exact model, and the provenance line on the page is drawn from it.

```
def citation(self, claim: dict[str, Any], sources: list[dict[str, Any]]) -> None:
    checked("Claim", claim)
    if redact_personal(claim["text"])[1]:
        raise StudyError("insufficient_evidence")
    allowed = {(s["source_id"], s["revision"]) for s in sources}
    for c in claim["citations"]:
        if (c["source_id"], c["revision"]) not in allowed:
            raise StudyError("insufficient_evidence")
        chunk = self.one(
            "SELECT body FROM study_chunks WHERE chunk_id=? AND source_id=? AND revision=?",
            (c["chunk_id"], c["source_id"], c["revision"]),
        )
        if (
            not (0 <= c["start"] < c["end"] <= len(chunk["text"]))
            or chunk["text"][c["start"] : c["end"]] != c["quote"]
        ):
            raise StudyError("insufficient_evidence")
```

`services/agent-core/agent_core/engine/study.py`, lines 1349–1365, revision df10b972. The check that every quote equals its stored span. A support call shows a passage was judged to back the item, and this shows the quote is real. Neither is proof that the item is right.

Capabilities

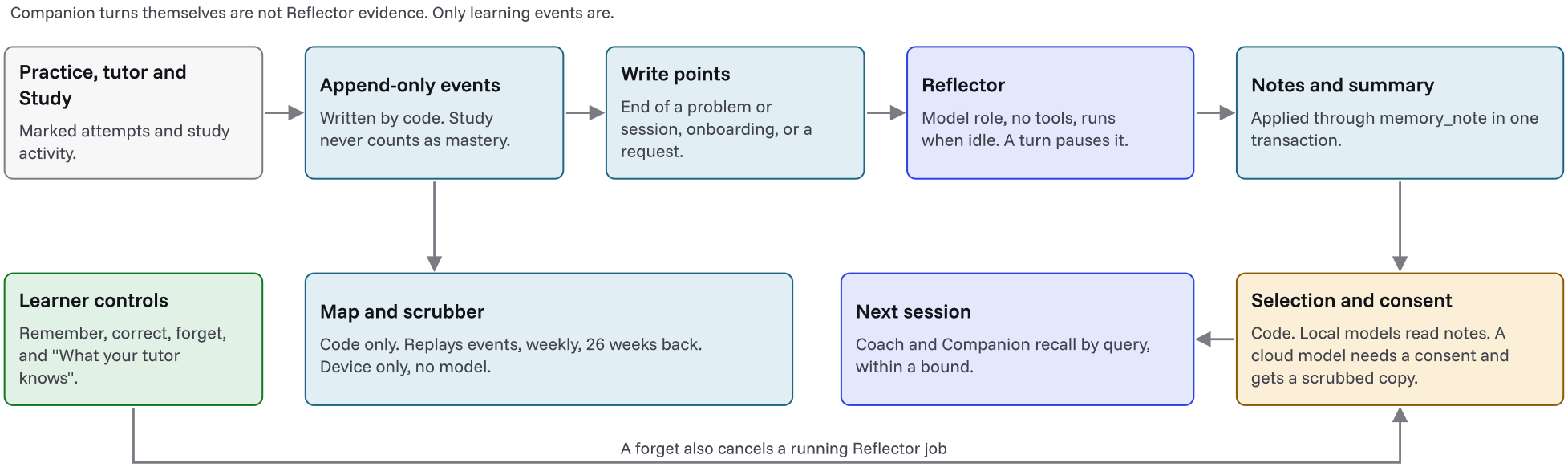
CAPABILITY	WHAT THE LEARNER SEES	HOW IT WORKS	STATUS
Material intake	PDF, Word, slides, text, code and data, images, audio, video. Up to 100 MiB, 1,000 pages, 120 minutes.	Sandboxed decode with page, slide or time locators.	Built, tested
Study tools	Summary, quiz, flashcards, notes, podcast, lesson plan, concept map, chapters, outline.	Model drafts and a second call checks. Code binds quotes and drops what is unsupported.	Built, tested
Quiz marking	A verdict, labelled "Checked" or "Judged, not checked".	Keys for choices, symbolic checker for numbers, judge plus blind check for written answers.	Built, tested
Ask this material	A cited answer or a refusal. Quiz help withholds the answer.	Code selects literal evidence and a verifier must pass every claim.	Built, tested
YouTube lecture	Paste a link and transcript, get an outline with "Open at m:ss". No player.	Jana fetches nothing. Code parses moments, the model writes cited outline items.	Built, tested
Research agent	Up to 3 questions, approve a brief, get a cited web pack.	Gateway checkpoints each sub-question. Host policy and address checks apply to every fetch.	Built, tested
Notes and saved	Own notes, session notes, saved questions.	Code stores. A model only selects exact excerpts.	Built, tested

Retrieval over materials is lexical, with no embeddings, so a paraphrase can miss. Long sources sample 24 sections. Study is online only on the browser and hosted routes: new jobs wait as "Waiting for the internet" and resume without new billing.

WHERE IT LIVES `services/agent-core/agent_core/engine/study.py` `services/agent-core/agent_core/engine/study_links.py` `services/agent-core/agent_core/engine/study_outline.py` `services/agent-core/agent_core/engine/research_chat.py` `services/cloud-gateway/cloud_gateway/study.py` `services/cloud-gateway/cloud_gateway/research.py` `services/study-worker/study_worker` `packages/ui/src/learn/StudyGenerate.tsx` `packages/client-core/src/study-workspace.ts` `services/agent-core/tests/test_d133_study_online_e2e.py`

7 Your tutor and learner memory

Each learner has one continuing conversation with a tutor that remembers them. Code fetches the facts and checks the wording, so progress numbers have to come from tools. The learner can see and change what is remembered.



Learner memory across sessions. Notes are written at four points only, and a forget always wins.

`LEARNER BENEFIT`

The tutor continues from last time and can explain why it personalises, and the learner stays in control.

`FLOW`

Events become notes at write points. A later session recalls them under a bound, filtered by route and consent.

`COMPONENTS`

Companion, Reflector, learner database, memory tools, consent switches, "What your tutor knows".

`CODE DECIDES, MODEL JUDGES`

Code: routing, recall, facts, actions, gates, what is stored. Model: wording, tool choice and proposing notes, which a filter can refuse whole.

Selection and consent, in code

```
def cloud_tools(names: Sequence[str], share_materials: bool, memory_online: bool =
False) -> list[str]:
    """The tools a cloud companion call is offered: never raw session recall (also
enforced in the handler),
    `memory_recall` only under `tutor_memory_online` (its answer scrubbed; MDL-25
    otherwise), `study_search` only under `study_materials`."""
    out = [n for n in names if n != "session_recall" and (n != "memory_recall" or
memory_online)]
    if not share_materials:
        out = [n for n in out if n != "study_search"]
    return out
```

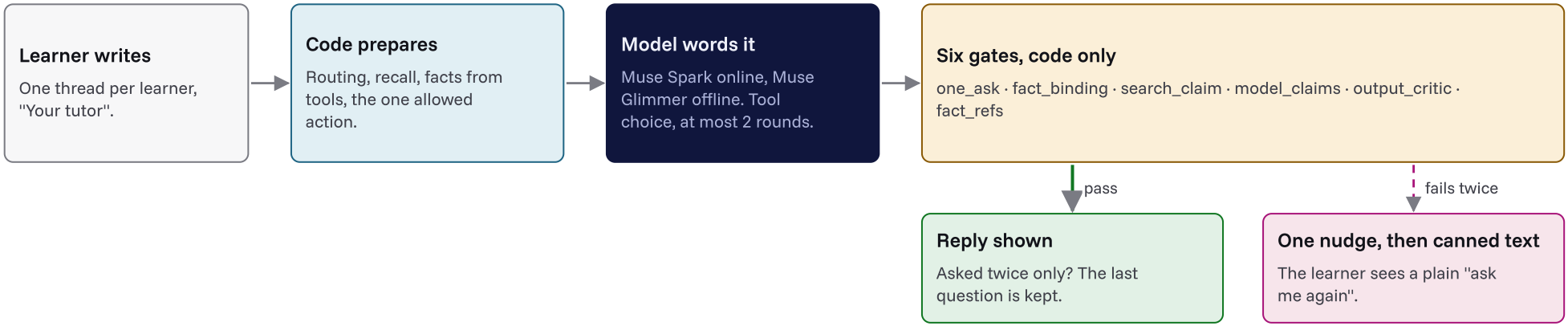
`services/agent-core/agent_core/engine/companion.py`, lines 700–707, revision df10b972. What a cloud Companion call is offered.

```
async def recall(self, args: dict[str, Any], context: CallContext) -> dict[str, Any]:
    online = self.cloud() and self.memory.memory_online()  # SPEC §19.6: under
tutor_memory_online, scrubbed
    if self.cloud() and not online:
        raise InvalidArguments("notes are not read for a cloud model")
```

`services/agent-core/agent_core/engine/memory_tools.py`, lines 137–140, revision df10b972. The handler refuses too, so the rule holds even if a model asks.

The Companion's gates

make_aid is refused unless the learner asked for a study tool, so a question does not start one.



Gates are heuristics on the wording, mostly in English.

Worked example. Real output of the fact-binding gate on draft replies, on this revision.

DRAFT REPLY FROM THE MODEL	GATE FINDS	WHAT HAPPENS
"These are two kinds of learning. Which one fits your dog?"	Nothing	Passes. "Two" is plain English here
"You have two cards due."	2, beside "card", no source	Refused unless a tool returned that number
"Your exam is in nineteen days."	19, beside "day", no source	Refused unless a tool returned that number
"It peaks at seventy degrees."	70, no source	Refused unless the material says it

DRAFT REPLY FROM THE MODEL	GATE FINDS	WHAT HAPPENS
"The behaviour comes first. Is that clear? What follows it?"	Two questions, after a nudge	Trimmed to "The behaviour comes first. What follows it?"

LOCAL-MODEL FIXES, IN THIS REVISION. Replies are fitted at a sentence boundary, keeping code-rendered facts intact. Companion requests apply the role's output limits. A local turn stops at 240 s with a plain line. The tutor answers questions about earlier messages from the conversation, and adds schedule facts only when they were asked about. A question it quotes back no longer counts as a second question. Tests: `test_companion_reply_limits.py`, `test_chained_line.py`.

Capabilities

CAPABILITY	WHAT THE LEARNER SEES	HOW IT WORKS	STATUS
Remember, correct, forget	Commands in chat, and a "What your tutor knows" page.	Patterns, no model. Free-text remember needs a local model, even with the online consent.	Built, tested
Personal tutor memory online	A switch, off by default.	Notes are scrubbed of capture, policy, secrets and profile terms. Matches are withheld.	Built, tested
Learner's map	Layers for mastery, freshness, mistakes, Study and notes, plus a time scrubber.	Code only. No model and no network.	Built, tested
Spaced review	Rate cards and see due counts.	FSRS-6. Study cards are never mastery evidence.	Built, tested
Encrypted export	A passphrase file, imported as a new profile.	Argon2id with AES-256-GCM. Transcripts need separate authorisation.	Built, tested

WHERE IT LIVES `services/agent-core/agent_core/engine/companion.py` `services/agent-core/agent_core/engine/companion_facts.py` `services/agent-core/agent_core/engine/memory_tools.py` `services/agent-core/agent_core/engine/companion_memory.py` `services/agent-core/agent_core/engine/reflection.py` `services/agent-core/agent_core/engine/learner_map.py` `services/agent-core/agent_core/memory/scrub.py` `services/agent-core/agent_core/store/exports.py` `packages/ui/src/learn/CompanionThread.tsx` `packages/ui/src/memory/KnowsPage.tsx` `packages/ui/src/progress/MapOverview.tsx` `services/agent-core/tests/test_companion_continuity.py` `services/agent-core/tests/test_companion_fact_binding.py` `services/agent-core/tests/test_companion_any_material.py` `services/agent-core/tests/test_w8_memory_online.py`

8 Curriculum

Study works on a learner's own material at adult and university level, with no curriculum needed. Separately, Jana has authored O-level packs. The development repository holds four, and the downloadable package includes three.

Each pack is data: skills, prerequisites, items, misconceptions and lessons, with answers recomputed by tools at build time.

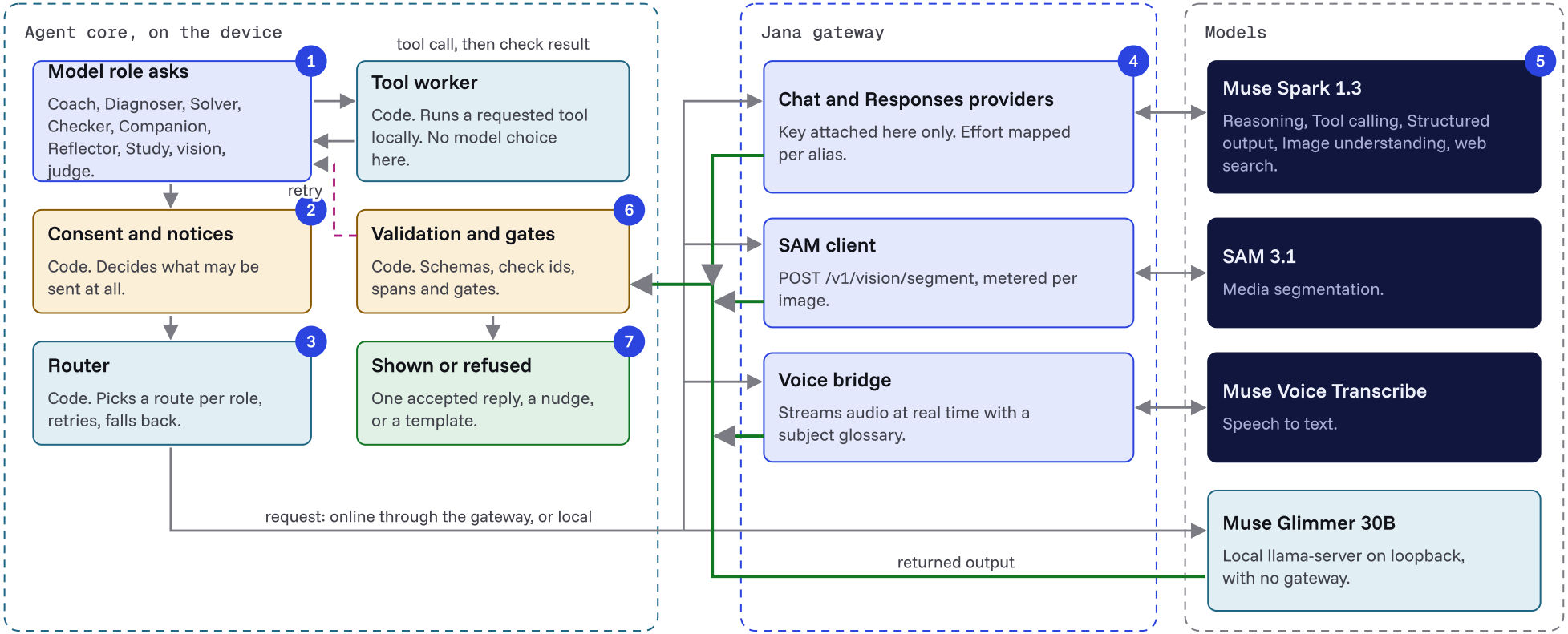
PACK	IN THE DOWNLOAD	VERSION	SKILLS	ITEMS	MISCONCEPTIONS	WORKED EXAMPLES	LESSONS	MOCK PAPERS
Mathematics	Yes	1.1.0	102	1,592	288	204	65	6
Physics	Yes	1.2.1	110	1,627	707	210	27	4
Chemistry	Yes	1.1.0	77	900	244	150	43	3
Download total			**289**	**4,119**	**1,239**	**564**	**135**	**13**
Accounting	No, development only	1.2.0	78	862	242	154	17	3
Development repository total			**367**	**4,981**	**1,481**	**718**	**152**	**16**

428 prerequisite edges connect the skills in the development repository. The content is AI-drafted, and the records inspected have no named human reviewer yet. A Biology subject code and tools exist with no pack.

WHERE IT LIVES `content/curriculum` `services/pack-builder/pack_builder` `scripts/build-submission.sh` `apps/studio`

9 How Meta's models are integrated

Muse Spark handles cloud reasoning and vision, Muse Glimmer local tutor inference, SAM 3.1 segmentation on a learner's own line, and Muse Voice Transcribe cloud speech recognition. Around each model, code decides what may be sent and what is kept.



Read the numbers in order: a role asks, consent is checked before anything leaves, the router picks a route, the provider calls the model, and the returned output goes through validation before anything is shown. When a reply asks for a tool, the engine runs it locally and sends the check result back in the next request. The Meta key exists only in the gateway. Capability names follow Meta's developer documentation.

Muse Spark

MODEL	`muse-spark-1.3`, Standard mode
CAPABILITIES	Reasoning, Tool calling, Structured output, Image understanding, web search
ENDPOINTS	`/v1/chat/completions`, `/v1/responses`
MEASURED	Guided maths turn from about 23 s to 12.8 s after effort tuning. Then median 7.2 s, p90 11.4 s over 20 seeded turns, with 30 of 30 turns keeping the rules (27 Oct)

Muse Glimmer

MODEL	Muse Glimmer 30B, open-weight, self-hosted
CAPABILITIES	Local inference, vision projector, DFlash draft
ENDPOINT	llama-server on loopback
MEASURED	4 Nov: 9 replies in 21.9 to 122 s and a photo in 37 to 85 s on an M4 Max. 9 replies in 22.8 to 49 s and a photo in 43 s on a 24 GB NVIDIA L4, peaking at 20.7 GB of GPU memory. 2.1 tokens a second on 8 vCPUs with no GPU

SAM 3.1

MODEL	`sam-3.1`, Segment Anything Model 3.1
CAPABILITY	Media segmentation
ENDPOINT	`/v1/responses`
MEASURED	Synthetic plant diagram: 5 of 5 parts named, 4 to 5 of 5 outlined, ready in about 15 to 20 s

Muse Voice Transcribe

MODEL	Muse Voice Transcribe
CAPABILITY	Speech to text
ENDPOINT	`wss://api.meta.ai/v1/asr/realtime`
MEASURED	Final transcript in 1.08 to 1.22 s on three recorded questions (27 Oct)

Muse Spark: role selection and requests

Each role names a tier in the router configuration. The Solver prefers the cloud and needs tools. The gateway then maps the requested effort per model alias and adapts the request to what Muse Spark accepts.

```
solver:
  tier: strong
  prefer: cloud
  reasoning: high
  max_iter: 12
  temperature: 0.2
  output: solution-plan.schema.json
  output_retries: 1
  prompt_mode: minimal
  requires:
    tools: true
    min_context: 6144
  priority: 6
```

`services/agent-core/agent_core/router/router.yaml`, lines 49–61, revision df10b972. The Solver's role entry.

```
effort = out.get("reasoning_effort")
if effort is not None and effort not in EFFORTS:
    out["reasoning_effort"] = "minimal"  # "none": Spark always reasons; minimal is
its lightest setting
⋮
out["store"] = False  # no stored completions at Meta
```

`services/cloud-gateway/cloud_gateway/providers/meta.py`, lines 47–49, 64, revision df10b972. Muse Spark always reasons, so "none" becomes minimal.

Jana uses Standard mode and sends `store: false` on supported requests. Provider retention follows the applicable Meta account settings and policies, and this flag alone does not establish Zero Data Retention (field reference, Zero Data Retention).

Muse Glimmer: local attachment and routing

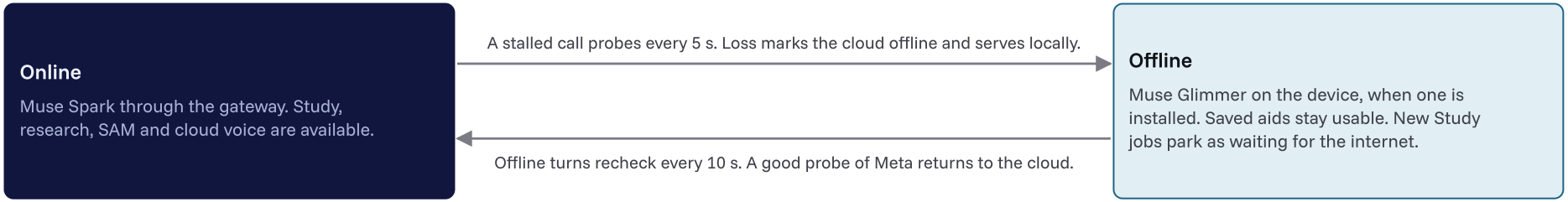
The launcher starts llama-server on loopback with pinned weights. The router marks the cloud offline after a failed probe and brings it back when a probe succeeds.

```
def runtime_arguments(model: Path, port: int, key_file: Path) -> list[str]:
    """llama-server's arguments for Glimmer: the GGUF's own chat template (`--jinja`),
the vision projector so offline
    photos work, and DFlash speculative decoding (30-45% faster generation on short
prompts, SPEC §7). Reasoning is
    left on, as Glimmer requires: the app sets its strength per request."""
    args = ["--model", str(model), "--alias", f"{MODEL},muse-glimmer", "--host",
"127.0.0.1", "--port", str(port),
            "--api-key-file", str(key_file), "--ctx-size", str(CONTEXT), "--parallel",
str(SLOTS),
            "--n-gpu-layers", "99", "--jinja", "--no-webui", "--log-disable"]  # fmt:
skip
    projector, draft = model.with_name(PROJECTOR), model.with_name(DRAFT)
    if projector.is_file():
        args += ["--mmproj", str(projector)]
    if draft.is_file():
        args += ["--model-draft", str(draft), "--spec-type", "draft-dflash", "--n-gpu-
layers-draft", "99",
                 "--spec-draft-n-max", "4"]  # fmt: skip
```

`scripts/jana_tools/hackathon_dev.py`, lines 55–67, revision df10b972. The local key file protects loopback and is not a Meta key.

```
if self.cloud_offline and self.network is not None and online:
    self.cloud_offline = False  # a probe found the network again after the failure
that marked it (W3)
if self.cloud_offline or not online:
```

`services/agent-core/agent_core/router/core.py`, lines 251–253, revision df10b972. A probe restores the cloud route.



Router behaviour, covered by a simulated power-cut test.

SAM 3.1: the segmentation path

Muse Spark returns a box per region. The core asks the gateway to sharpen them, and the gateway meters the call and sends the prompts to SAM. A failed or empty result leaves the box as Spark read it.

```
def body(self, image_url: str, prompt: str, *, one_bit: bool) -> dict[str, Any]:
    return {
        "model": self.spec.model,
        "input": [{"type": "message", "role": "user", "content": [
            {"type": "input_text", "text": prompt}, {"type": "input_image", "image_url": image_url}]}],
        # Values are strings: a JSON boolean is a 400.
        "metadata": {"mask_encoding": "one_bit" if one_bit else "lossless", "include_confidence": "true"},
        "store": False,
```

`services/cloud-gateway/cloud_gateway/providers/sam.py`, lines 176–183, revision df10b972. The request as built by the gateway. Nothing logs the image, the prompts or the masks.

Voice Transcribe: streaming

The gateway opens the realtime socket and sends the setup frame first, then 24 kHz audio at real time. Each partial replaces the last. Words spoken back come only from accepted turns.

```
"authorization": {"accessToken": "Bearer " + key.reveal()},
"model": voice.model,
"audioEncoding": "PCM_24KHZ",
"mode": "ENDPOINTING",
"partialMode": "CUMULATIVE",
"emitAudioProgress": False,
```

`services/cloud-gateway/cloud_gateway/providers/meta_speech.py`, lines 87–92, revision df10b972. The key is read from custody here and is never logged. It does not reach the device.

Where consent is checked

```
def guard(self, conn: Any) -> str:
    pid = self.profile()
    if conn is not self.connection() or pid is None:
        raise StudyCloudError("consent_required")
    if not {"study_materials", "cloud_tier"} <= self.layer.features_on():
        raise StudyCloudError("consent_required")
    if not subscription.covers(self.layer.subscription(), pid, self.layer.clock_ms()):
        raise StudyCloudError("allowance_exhausted")
```

`services/agent-core/agent_core/net/study.py`, lines 54–61, revision df10b972. Study refuses to go online without both consents and an allowance, before any source is sent.

Which capability uses which model

CAPABILITY	MUSE SPARK	MUSE GLIMMER	SAM 3.1	VOICE TRANSCRIBE	OUTSIDE META
Guided tutor turn	All roles, online	Lean turn, offline			
Photo of working	Reads lines, returns boxes	Reads lines, offline	Outlines the wrong line		YuNet refuses faces first, Tesseract for OCR
Labelled diagram drill	Names parts, returns boxes		Outlines each part		

CAPABILITY	MUSE SPARK	MUSE GLIMMER	SAM 3.1	VOICE TRANSCRIBE	OUTSIDE META
Study aids and quiz	Drafts, then a separate support check				
Ask this material	Answers, then a verifier passes each claim				
Research agent	Plans and searches				
Your tutor	Wording and tool choice, online	Wording and tool choice, offline			
Speaking to the tutor	Answers the question			Streams the transcript	Kokoro-82M reads accepted words

What happens without each model

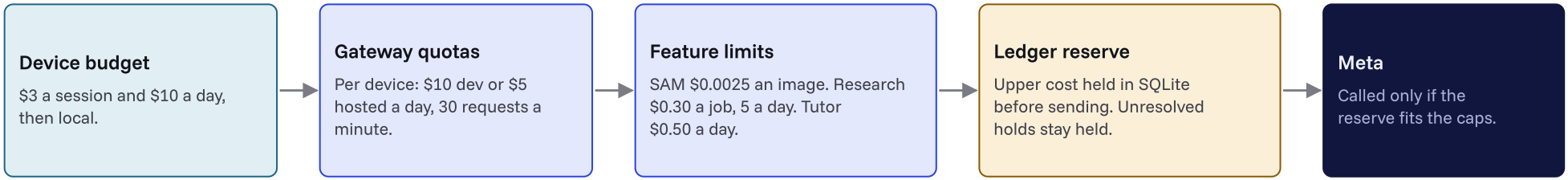
COMPONENT	IF IT IS MISSING	STATUS
Muse Spark	Router uses Glimmer. Study and research wait.	`Built, tested`
Muse Glimmer	Online only, or authored content only when offline.	`Verified on M4 Max and NVIDIA L4` `Built, partial` packaged app
SAM 3.1	Spark's box is kept, with no outline.	`Built, tested`
Muse Voice Transcribe	Speech input is unavailable. Off on the hosted deployment.	`Built, tested`
Kokoro-82M, outside Meta (Apache-2.0)	Speech synthesis. Degradation not confirmed.	`Built, tested`
YuNet, outside Meta (MIT)	Photo processing fails closed.	`Built, tested`

Tests: `test_meta.py` , `test_sam.py` , `test_meta_speech.py` , `test_m11c_router.py` , `test_m11f_vision.py` , `test_w3_power_cut.py` . Passed, 4 Nov. Measurements are dated, and some were taken on earlier commits.

WHERE IT LIVES `services/cloud-gateway/cloud_gateway/providers/meta.py` `services/cloud-gateway/cloud_gateway/providers/meta_responses.py` `services/cloud-gateway/cloud_gateway/providers/sam.py` `services/cloud-gateway/cloud_gateway/providers/meta_speech.py` `services/cloud-gateway/cloud_gateway/providers/kokoro_speech.py` `services/cloud-gateway/cloud_gateway/segment.py` `services/cloud-gateway/cloud_gateway/custody.py` `services/agent-core/agent_core/router/router.yaml` `services/agent-core/agent_core/router/core.py` `services/agent-core/agent_core/router/probe.py` `services/agent-core/agent_core/engine/photo_regions.py` `services/cloud-gateway/tests/test_meta.py` `services/cloud-gateway/tests/test_sam.py` `services/cloud-gateway/tests/test_meta_speech.py` `services/agent-core/tests/test_m11c_router.py` `services/agent-core/tests/test_m11f_vision.py`

10 Spending controls

Every cloud call has its largest possible cost reserved before it is sent. Limits sit on the device, per learner device, per feature and overall, and the service stops at a fixed date.



$10 dev or $25 hosted a day, $120 total each, stops 9 Nov 00:00 UTC.

Each layer can stop a call. A browser host fails closed without caps and a budget header. Muse Spark is priced at $1.25 input, $0.15 cached input and $4.25 output per million tokens.

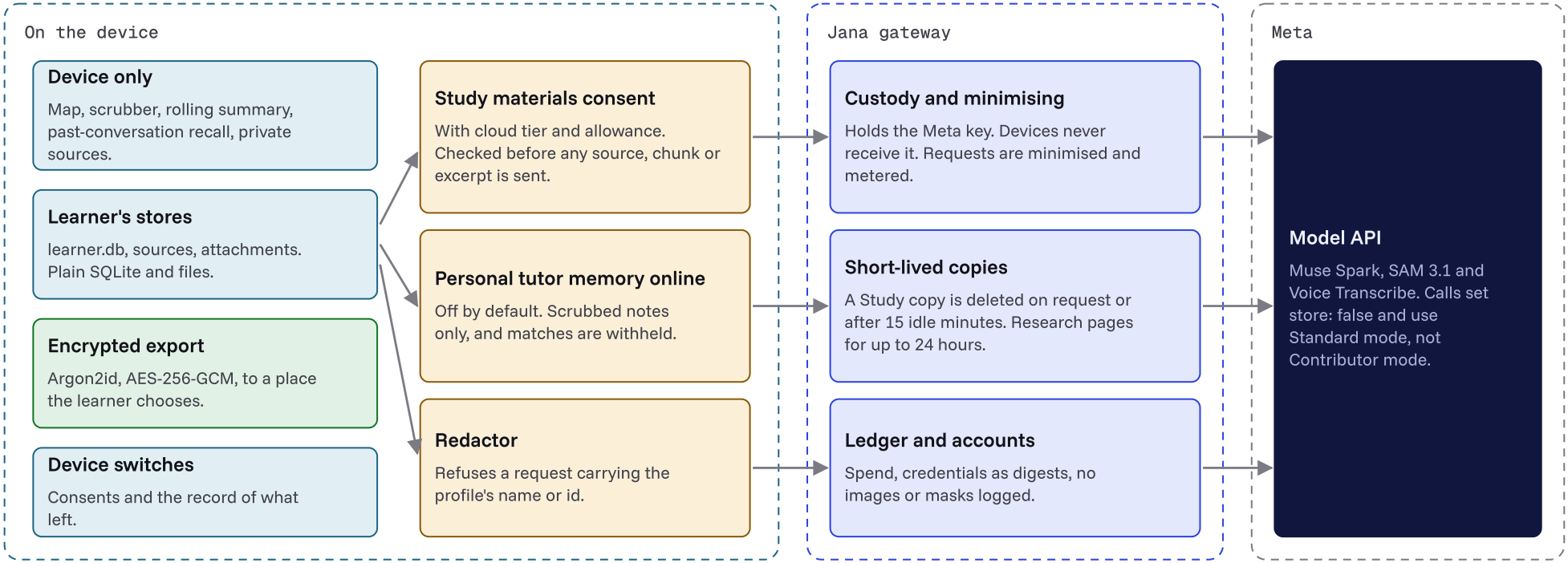
WHAT IS PROVED	TEST	RESULT
Parallel connections cannot spend the same headroom	`test_parallel_connections_cannot_spend_the_same_headroom`	Passed, 4 Nov
Unknown usage keeps the upper bound	`test_failures_and_missing_usage_keep_the_upper_bound`	Passed, 4 Nov
Holds left by a dead gateway are recovered	`test_holds_left_in_flight_by_a_dead_gateway_become_releasable_at_startup`	Passed, 4 Nov

Ledgers are per storage folder, so the development and hosted gateways together could reach $240, with each capped at $120.

WHERE IT LIVES `services/cloud-gateway/cloud_gateway/inference_budget.py` `services/cloud-gateway/cloud_gateway/holds.py` `services/cloud-gateway/cloud_gateway/quotas.py` `services/cloud-gateway/cloud_gateway/defaults.toml` `services/agent-core/agent_core/router/budget.py` `services/cloud-gateway/tests/test_inference_budget.py` `services/cloud-gateway/tests/test_ledger_leaks.py`

11 Privacy and storage boundaries

Learner data lives on the device in plain files, one set per learner. Each kind of data that goes online has a named consent, and the map, summaries and past-conversation recall never leave. Jana is for adults aged 18 and over.



Amber boxes are code checks. The Standard mode and `store: false` settings describe what Jana requests. Retention at Meta follows the account's settings and policies, and Zero Data Retention is not established here. Meta's documentation lists web search among the features disabled under it, and the research agent uses web search. Device-only data has no route out, and an export goes only where the learner chooses. In the hosted deployment the core runs on a server, so "the device" means that server.

What leaves, and what it needs

DATA	GOES TO	NEEDS
Source file, chunks, focus, prompt	Gateway, Muse Spark with `store: false`	Study materials consent, cloud tier
Written answer, question, cited passages	Same	Same
Podcast passage, up to 600 characters	Gateway speech	Same, plus the speech setting
Research chat and approved brief	Gateway, web search, fetched hosts	Same, plus a first-use notice
Companion message, filtered history, learner block	Gateway	Cloud tier and a first-use notice

DATA	GOES TO	NEEDS
Material excerpts in the Companion	Gateway	Adds Study materials consent
Scrubbed notes and evidence lines	Gateway	Adds Personal tutor memory online
Map, scrubber, rolling summary, past-conversation recall, private sources	Nothing	Device only
Passphrase archive	A place the learner chooses	Learner action

Where the data lives

STORE	HOLDS	ENCRYPTED
`device.db`	Switches, record of what left, outbox, profile index	No
`profiles/*/learner.db` and files	Learner record, transcripts, sources, attachments	No
`traces/traces.db`	Redacted traces	No
OS keychain, or owner-only files	Service credential	OS-protected. Browser and hosted use files
`gateway.sqlite3` , `operations.sqlite`	Credentials, token digests, ledger, research jobs for 24 hours	No
`accounts.sqlite3` on the hosted deployment	Scrypt password hashes and session digests	Digests only
Exports	Learner archives	Argon2id, AES-256-GCM

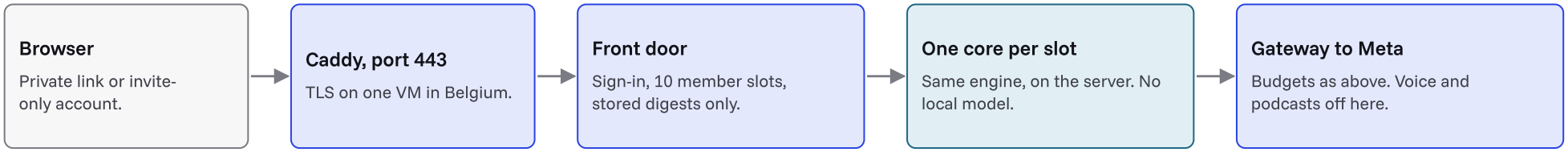
A redactor blocks any request carrying the profile's name or id. Profiles have no PIN, so anyone with the computer can open them.

WHERE IT LIVES `services/agent-core/agent_core/store/consents.py` `services/agent-core/agent_core/net/study.py` `services/agent-core/agent_core/net/redaction.py` `services/agent-core/agent_core/net/policy.py` `services/cloud-gateway/cloud_gateway/minimise.py` `services/agent-core/agent_core/memory/scrub.py` `services/agent-core/agent_core/store/profile_stores.py` `services/agent-core/agent_core/engine/profile_scope.py` `apps/desktop/src-tauri/src/keychain.rs` `scripts/jana_tools/hosted_accounts.py` `packages/ui/src/settings/OnlineFeatures.tsx` `services/agent-core/tests/test_m10a_isolation.py` `services/agent-core/tests/test_w8_memory_online.py`

12 Where it runs

The same core runs in three places. The browser host is for local runs, the hosted deployment needs no install, and the desktop shell is the intended product form, which has no complete release path yet.

HOST	USED FOR	MODEL ROUTES	STATUS
Browser host	Local runs on macOS and Linux. Vite serves the client and starts the core, with a local gateway. Started from cold with the network cut on Linux.	Muse Spark, Muse Glimmer, SAM, voice	`Built, tested`
Hosted deployment	Use without an install. One VM in Belgium, linked from getjana.app.	Muse Spark and SAM. No Glimmer, voice or podcasts	`Built, partial`
Desktop shell	The product form. Tauri with OS keychain, sandboxed sidecars and a launch plan for the local model. A debug build reached a local reply. A release build's core refuses to start until a release-signed agent bundle exists.	All, by design	`Built, partial`



The hosted deployment. Access is by private link or an invite-only account, and only digests are stored.

The hosted deployment cannot work offline. The browser host must start with a gateway present and can lose it afterwards. Cloud calls stop on 9 November 00:00 UTC by configuration.

WHERE IT LIVES `scripts/jana_tools/hackathon_dev.py` `scripts/jana_tools/hackathon_app.py` `scripts/jana_tools/hosted.py` `scripts/jana_tools/hosted_accounts.py` `scripts/hosted` `services/cloud-gateway/deploy/meta-hosted.toml` `apps/desktop/src-tauri/src/supervisor.rs`

13 Security and quality

Files, pages and model output are treated as untrusted. Each program gets narrow access, and contracts, snapshots and tests fail the build when something drifts.

CONTROL	WHAT IT DOES	WHERE	STATUS
Key custody	Only the gateway holds the Meta key. Child processes get a filtered environment.	`services/cloud-gateway/cloud_gateway/custody.py`	Built, tested
Device auth	Ed25519 challenge and 15-minute tokens.	`services/cloud-gateway/cloud_gateway/auth.py`	Built, tested
Local API guard	Launch token on loopback.	`services/agent-core/agent_core/api/security.py`	Built, tested
Outbound control	Allow-list and a ledger of what left. TLS pins and real hosts are not set yet.	`services/agent-core/agent_core/net/policy.py`	Built, partial
Untrusted text	Pattern-based injection handling on materials and fetched pages.	`services/agent-core/agent_core/prompts/untrusted.py`	Built, tested
Sandboxes	Seatbelt, bubblewrap, Landlock, AppContainer, seccomp, depending on platform. The speech sidecar is unsandboxed on Windows. `sec06_sandbox_escape_suite` , `tol26_sandbox_denies_io` : Passed, 4 Nov on macOS. The sandbox and platform-hardening tests also passed on a clean Linux VM: 39 passed, 1 macOS-only skipped.	`services/agent-core/agent_core/tools/worker/sandbox.py`	Built, tested
Signing	Ed25519 hierarchy for packs and updates. Only a test root exists.	`services/signing-service`	Built, partial
Supply chain and boundaries	Dependency reviews, licence rules, 35 import contracts, scans in CI.	`deny.toml` `.importlinter`	Built, tested
Hosted access	Link digests, httponly cookie, scrypt passwords, attempt limiter.	`scripts/jana_tools/hosted.py`	Built, tested

Tests and evidence

EVIDENCE	MEASURE	LATEST RESULT
Agent core tests	309 test files	Full run on this revision, 4 Nov: 3,644 passed, 44 skipped, 1 failed. The failure was a worker-availability error in `test_working_paste.py` during that long run, and the test passes when run alone.
UI tests	106 test files	817 passed, 4 Nov
Client core tests	49 test files	418 passed, 4 Nov
Other Python tests	Gateway 32, scripts 61, pack builder 22, tool worker 16 files and others	Run in CI
Desktop	50 TypeScript test files, 118 Rust test attributes	Run in CI
Contracts	109 schemas, 1,106 corpus files. `jana_tools codegen --check` fails CI on drift.	Lock checked 4 Nov
Evals	18 suites locked by hash. Safety 102 scenarios, marking 150 cases, 60 prompt snapshots, 29 rendered worksheets, 10 journeys.	Scripted suites run in CI. Some evidence predates Muse: see Part 14
CI	4 workflows, 12 jobs in the main pipeline	See the repository's Actions page

WHERE IT LIVES `evals/suites.toml` `.github/workflows/ci.yml` `services/agent-core/tests/test_m2b_sandbox.py`

14 Status and limits

Jana works end to end on real models, and most of the engineering is tested. It has not yet been shown to improve learning, and it is not a signed, released product.

Real limits

- **No learning-gain study.** None has been run, and no outcome is claimed. A simulated multi-day journey is covered by regression tests.
- **No release path.** Signing uses a test root, hosts are placeholders and TLS pins are empty. Browser and hosted cores run as non-release.
- **No encryption at rest** for learner databases, and no profile PIN.
- **Evidence predates Muse in places.** Prompt snapshots cover earlier local models, some live and perception runs predate this stack, and no model bake-off is recorded. The router also still lists earlier local candidates, used only if one is already installed.
- **Curriculum review.** Packs are AI-drafted and answers are recomputed by tools, without recorded human review.
- **Thin content in new features.** 10 root-gap links, 8 predict scenarios, few figures, and no grade bands or diagnostic anchors.
- **The verifier is the same model family as the drafter** for Study, and the device trusts the gateway's support filter, rechecking only the spans.
- **Local tutoring needs GPU-class hardware.** Text tutoring, guided tutoring and photo reading work on Muse Glimmer on an Apple M4 Max, at 22 to 122 seconds per reply, and on a 24 GB NVIDIA L4 in the cloud, at 23 to 49 seconds. It peaked at 20.7 GB of GPU memory. With no GPU it is not usable, and the learner gets a plain message after 240 seconds. No laptop other than the M4 Max is tested, including typical 32 GB machines.
- **Local role limits are longer.** A delegated role on the local model gets three times its limit: Checker 30 s, Diagnoser 90 s, Solver 180 s. On the lean local class the Checker is skipped when code has already marked a line wrong. A slower machine can still hit these limits, and then falls back.
- **No release desktop app.** A debug build works with a test-signed manifest and a manual override. A release build needs an agent bundle signed under a real release root, a pinned llama.cpp, and real hosts and pins. A speed test under load can also drop a machine to content-only, with no way to retest in the interface. No Windows build has been made.
- **Study does not generate offline,** and uploads made offline wait for the internet. This is a deliberate setting.
- **Service end date.** Cloud calls stop on 9 November 00:00 UTC by configuration, and the Meta credit's real end date is unconfirmed.
- **One person holds every role,** and `CODEOWNERS` still names a service that no longer exists.

Planned architecture work

A proposed order drawn from the limits above. None of this is built unless the row says so, and it is not a commitment. Learning evidence, content review and new curricula matter as much, but they are not architecture changes.

ORDER	CHANGE	WHERE	WHAT IT UNLOCKS
1	**Release trust chain.** A real signing root, a signed agent bundle, a pinned llama.cpp, real hosts and TLS pins, then Apple Developer ID signing and notarisation. A debug build and the manifest checks exist.	`apps/desktop`, runtime policy	A packaged app that a learner can install and that refuses tampered parts.

ORDER	CHANGE	WHERE	WHAT IT UNLOCKS
2	**Hardware-aware local runtime.** Read memory and GPU at start and say early if the model will not run well. Use fewer inference slots and less context on smaller machines, and add a way to rerun the speed test. Today the slots are fixed at 4.	Runtime launcher, router, desktop setup	A clear answer before a long wait, and a chance of running on smaller laptops.
3	**Protected learner data.** Encryption at rest for learner databases and a profile PIN.	Agent core store, profile layer	Real study records can live on a shared computer.
4	**Cold start with no gateway.** The browser host needs a local gateway process at start, even when no cloud call will be made. Make it optional when only the local model is used.	`hackathon_app.py`, connectivity	A simpler offline setup.
5	**Windows.** A signed build on the same trust chain, with the speech sidecar sandboxed like the other tools. Windows sandbox tests already run in CI.	`apps/desktop`, voice launch	A second platform.
6	**Stronger Study checks.** A verifier independent of the drafter, embeddings over materials, and local generation where hardware allows.	Gateway, Study pipeline	Better recall, a stronger check, and aids made offline.

A Code map

The downloadable package is a subset of this repository. It leaves out `apps/studio`, `apps/prototype-s0`, `services/ingest`, the Accounting pack, CI configuration and release tooling.

DIRECTORY	WHAT IT HOLDS
`apps/desktop`	React client and Tauri shell
`apps/studio`	Authoring studio
`apps/prototype-s0`	Frozen first prototype
`packages/client-core`	Client state and streams
`packages/contracts-ts`	Generated types
`packages/i18n`	Text catalogues
`packages/renderers`	SVG figures
`packages/ui`	Components and gallery
`services/agent-core`	Engine, API, router, stores
`services/cloud-gateway`	The boundary with Meta
`services/tool-worker`	Maths and science checks
`services/perception-worker`	Photo cleaning and OCR
`services/study-worker`	Document extraction
`services/pack-builder`	Curriculum authoring tools
`services/signing-service`	Pack and update signing
`services/ingest`	Telemetry and crash intake, not material ingest
`content`	Agent roles, templates, curriculum
`contracts`	Schemas and registers
`evals`	Eval suites, snapshots, results
`skills`	Teaching skills
`scripts`	Tooling, CI, hosted deployment
`assets`	Logos

B Glossary

TERM	MEANING
Aid	A study tool made from a source: summary, quiz, flashcards and so on.
Companion	The persistent tutor thread, shown as "Your tutor".
Consent	A named switch that allows a kind of data to go online.
FSRS	The spaced repetition scheduler used for reviews.
Gate	A code check a draft reply must pass before the learner sees it.
Gateway	Jana's own service that holds the Meta key and the spending limits.
Glimmer	Muse Glimmer 30B, Meta's open-weight model running locally.
Pack	A signed bundle of curriculum for one subject and level.
Reflector	The role that turns activity into tutor notes.
SAM 3.1	Meta's Segment Anything Model 3.1, used to outline regions on photos.
Skill	The smallest unit of knowledge a pack tracks.
Slot	One isolated learner space on the hosted deployment.
Spark	Muse Spark, Meta's hosted reasoning model.